A Twin-Gated Control Loop for Campus Wi-Fi: Verifying SDN and LLM-Proposed Actions in a Digital Twin Before They Reach the Network

Aggimalla Abhishek, N. Likhith Naik, Sundaram, Sambhav Mishra, Bikram Hawaldar
Indian Institute of Information Technology Dharwad, India

Jagadeesha R B
Dept. of Electronics and Communication Engineering
Indian Institute of Information Technology Dharwad, India

***Abstract*—A software-defined network (SDN) controller can react to congestion, access-point (AP) failures and interference in campus Wi-Fi, but a change applied directly to the live network shows its effect only afterwards. We present a control loop in which no action, whether proposed by a heuristic optimizer or by a large language model (LLM) that translates an operator's intent, reaches the network without an accepted verdict from a digital twin. The twin is an analytical model of per-AP airtime sharing, co-channel interference and M/M/1/K queueing. An executor applies accepted actions, watches the live KPIs and rolls back regressions. On an emulated campus (Mininet-WiFi and Ryu; 4 APs, 20 stations) we compare plain SDN (V1), heuristics without the twin (V2) and the full system (V3) in three scenarios and three seeds each (27 runs). The twin predicts per-flow throughput within 4.5% and answers in a median 0.61 ms. In a lecture flash crowd both acting variants cut the loss of the affected flows from 43.9% to under 10% and more than double their throughput. The hypothesis that V3 recovers 30% faster than V1 stays inconclusive, because plain SDN never recovered. The twin rejected few actions: most harmful ones were accepted, and the watch-and-rollback step, which itself failed 3 times, gave the main protection. An LLM behind schema validation translated 30 of 30 test intents correctly. We report these negative results together with the code, data and scripts that reproduce them.**

***Index Terms*—digital twin, software-defined networking, Wi-Fi, large language models, intent-based networking, safe automation**

I. Introduction

Campus Wi-Fi is never still: students move between rooms, a lecture fills a hall within minutes, an AP fails, and neighbouring APs on one channel slow each other down. An SDN controller can react by steering clients to a less loaded AP, changing an AP's channel or prioritizing a traffic class. Reacting is risky, because a controller that applies a change directly learns its effect only afterwards, and the risk grows as more decisions are automated, whether by an optimizer or by an LLM that turns a goal such as "video calls in the lab must stay smooth" into policy. LLM output can be wrong, incomplete or unsafe, and nothing checks it before it takes effect.

We make a *digital twin* of the network the gate that every change must pass. The twin mirrors the live state from telemetry and predicts the effect of a candidate action on throughput, latency, loss and fairness before it is applied. We ask whether this makes an AI-driven controller both more effective and safer than acting directly, and whether an LLM can drive it from natural language. We test three hypotheses: **H1** the full system (V3) recovers from congestion at least 30% faster than plain SDN (V1); **H2** the twin blocks actions that would hurt the KPIs and that an optimizer acting directly (V2) would apply; **H3** an LLM with schema-constrained output translates at least 20 of 30 intents correctly, and no invalid configuration reaches the network.

Contributions. (i) A reproducible, fully virtual wireless SDN testbed with seeded campus scenarios and an explicit model of co-channel interference, which the emulator does not produce itself. (ii) An analytical twin of airtime sharing and queueing, validated against the testbed, fast enough for a 5 s control loop. (iii) A closed loop (propose, verify, apply, watch, roll back) in which the twin's accepted verdict is enforced in the code that applies changes. (iv) An LLM intent pipeline in which the model only translates text into a validated policy; deterministic code compiles it into bounded actions. (v) An evaluation of three controllers over 27 runs that reports what did not work.

II. Related Work

Network digital twins. Almasan et al. [1] describe the network digital twin as an accurate, data-driven model of a real network, used chiefly to answer "what if" questions before a change. We share the split between a mirror of the live state and a performance model, but our model is analytical, needs no training data and can be explained, and we use the twin as a *safety gate*: no action is applied without its accepted verdict. *Learned performance models.* RouteNet [2] uses a graph neural network to predict per-path delay, jitter and loss and to score routing candidates. We follow its use of a fast model inside the loop and its relative-error reporting, but our bottleneck is the shared Wi-Fi medium, which we model explicitly; a learned surrogate is future work. *LLM-*

based intent networking. Mekrache et al. [3] manage the intent life cycle in natural language on a 5G testbed. We adopt natural language as the operator interface, but trust the model less: its output must pass a JSON schema, is compiled by deterministic code and is simulated in the twin, and the model has no tool that changes the network. *Programmable WLANs.* Odin [4] makes enterprise WLANs programmable through a per-client virtual AP, because clients, not the infrastructure, choose their AP. We steer clients by explicit re-association, which costs a reconnection that the twin accounts for. *Emulation.* We build on Mininet [5], Mininet-WiFi [6] and OpenFlow [7] with the Ryu controller. None of these works combines a wireless twin, natural-language control and a verification step before every action.

III. Problem Scenario and System Model

A. Scenario

The campus has four zones, each served by one AP: a lecture hall (ap1, channel 1), a lab (ap2, channel 6), a corridor (ap3, channel 11) and a library (ap4, channel 1), two OpenFlow switches and 20 stations carrying video, web and bulk traffic. ap1 and ap4 share channel 1 but are 50 m apart. An AP carries about 4.6 Mbit/s downlink on our testbed. Each 600 s scenario has one disruption: a *lecture flash crowd* (ten students walk into the hall from 120 s, video starts at 210 s), an *AP failure* (ap2 goes down at 240 s) and *co-channel interference* (ap3 is forced onto channel 1 at 180 s). The problem is to restore service quickly using only allow-listed actions, without making things worse for users.

B. System model

Fig. 1 shows the architecture. The testbed streams AP, station and flow telemetry to a time-series store. The twin rebuilds its state s from the last 10 s of telemetry at each refresh (no accumulated state, so it cannot drift from the network); an AP silent for 5 s counts as down. The state holds every AP (position, channel, up/down, utilization), station (AP, zone) and flow (measured throughput, latency, loss, queue, rate limit).

An *action* a is one of seven allow-listed types (reroute, QoS queue, rate limit, client steering, transmit power, channel change, AP admin state), each with an impact class: client steering and QoS are applied when the twin accepts them, while a channel change or taking an AP down is *high impact* and always waits for an operator. The measured KPI vector of a set of flows is $\mathbf{k} = (T, L_{95}, \bar{\ell}, J)$: mean throughput, 95th-percentile video latency, mean loss and Jain's fairness index [8] of the APs' client counts.

IV. Mathematical Formulation

A. Analytical twin

Capacity. An AP i that is up offers $C = 4.6$ Mbit/s, reduced by every up AP $j \neq i$ on its channel:

$$C_i = \frac{C}{1 + \sum_{j \neq i,\ \mathrm{ch}(j)=\mathrm{ch}(i)} w(d_{ij})}, \tag{1}$$

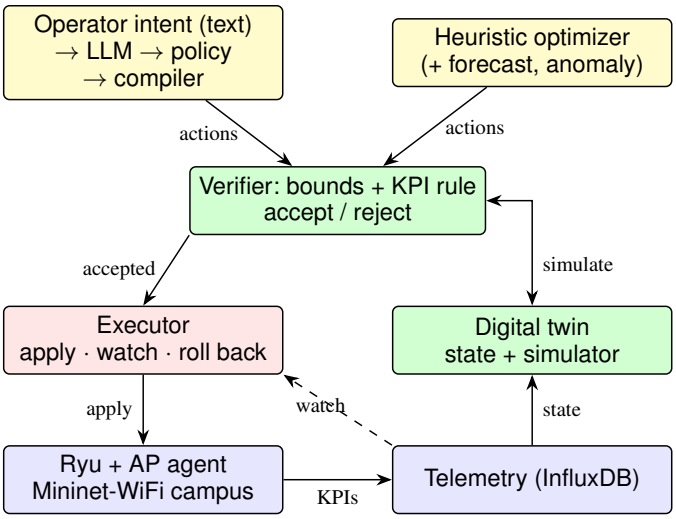


Fig. 1. Architecture. Proposals come from a heuristic optimizer or from an LLM-compiled policy. Only actions that the verifier accepts reach the executor, the only component that changes the network.

where d_{ij} is the distance between the APs, $w(d) = \min\{1, \max\{0, (d_2 - d)/(d_2 - d_1)\}\}$, $d_1 = 30$ m (a same-channel AP this close takes a full share) and $d_2 = 60$ m (from here it no longer interferes); the same model drives the testbed's emulated interference.

Sharing. Each AP serves three queues q in strict priority (priority, best effort, background). Within a queue, flow f with demand λ_f (its source rate, cut to its rate limit) receives the max-min fair share of the capacity left to the queue,

$$x_f = \min(\lambda_f, \theta_q), \qquad \sum_{f \in q} \min(\lambda_f, \theta_q) = R_q, \tag{2}$$

where R_q is C_i minus what higher queues take. An inelastic flow (video) loses $\ell_f = 1 - x_f/\lambda_f$ plus a base loss; an elastic one slows down and sees only the base loss.

Latency. Each queue is an M/M/1/K queue [9] at load $\rho_q = \sum_{q' \preceq q} \sum_{f \in q'} \lambda_f / C_i$ (its own demand plus that of the queues served before it):

$$d_q = d_0 + s\, L(\rho_q, K), \quad L(\rho, K) = \frac{\rho}{1-\rho} - \frac{(K+1)\rho^{K+1}}{1-\rho^{K+1}}, \tag{3}$$

with $L(1, K) = K/2$, $d_0 = 0.8$ ms, $s = 2.5$ ms and $K = 6$. A flow without a working AP is dead (no throughput, 100% loss). Network fairness is

$$J = \frac{(\sum_i n_i)^2}{N \sum_i n_i^2}, \tag{4}$$

over the client counts n_i of the N APs that are up. The simulator maps a state and an action set to predicted KPIs, $\hat{\mathbf{k}} = f(s, a)$.

Algorithm 1 One tick of the twin-gated control loop (every 5 s)

finish due watches: if a KPI regressed (S4) **roll back**, else **keep**
if an earlier change is still being watched **then**
  return
end if
$s \leftarrow$ twin state from the last 10 s of telemetry
$P \leftarrow \textsc{Propose}(s)$ (heuristics, or compiled LLM policy)
for each proposal $p \in P$ **do**
  $v \leftarrow \textsc{Verify}(s, p)$: bounds (S1), simulate $\hat{\mathbf{k}} = f(s, p)$, rule (6); record v
  if v accepted **and** p is not high impact **and** at most 3 applied this tick **then**
    $\textsc{Apply}(p)$ through the executor; start its watch
  else if v accepted **then**
    queue p for operator approval (S3)
  end if
end for

B. Safe control problem

Let t_d be the time of the disruption and $\mathcal{U}(t)$ the utilization of the affected AP. The *time to recover* is

$$T_{\mathrm{rec}} = \min\{t \geq t_d :\ \mathcal{U}(\tau) < 0.8 \text{ and } L_{95}(\tau) \leq 50\,\mathrm{ms} \text{ for all } \tau \in [t, t + 15\,\mathrm{s}]\} - t_d. \quad (5)$$

The controller chooses at each 5 s tick a set of actions a_t to minimize T_{rec} subject to: (S1) $a_t \in \mathcal{A}(s_t)$, the hard bounds (a steer moves at most 30% of an AP's clients, only stations on that AP, only to an AP where the predicted signal is at least -75 dBm; a transmit-power step is at most 3 dB; the last AP of a zone is never taken down); (S2) $\mathrm{accept}(s_t, a_t)$ holds, with δ_k the signed improvement of KPI k in percent (positive is better, for latency and loss lower is better) between $\mathbf{k}(s_t)$ and $\hat{\mathbf{k}} = f(s_t, a_t)$:

$$\text{accept} \iff \Big(\forall k :\ \delta_k \geq -5\Big) \ \vee \ \Big(\exists k :\ \delta_k > 5\Big); \quad (6)$$

(S3) a high-impact action is applied only after an operator approves it; (S4) for 30 s after an action the live KPIs are measured, ignoring the first 10 s; if any KPI is worse than before by more than 10% *and* by more than its measured noise floor, the action is rolled back.

V. Proposed Methodology

Algorithm 1 is one tick of the loop. The heuristics propose two kinds of changes: *client steering* (an AP at or above utilization 0.8 sheds up to 30% of its clients to APs below 0.6, strongest predicted signal first) and *channel change* (a congested AP that shares its channel moves to the least loaded one). While a change is being watched nothing new is proposed, so effects are not mixed up, and a proposal identical to a just-rolled-back one is not repeated for 120 s.

Safety enforcement. The executor is the only component that changes the network. It applies a set only if the action is in its append-only ledger with an accepted verdict, has the required approval, has not been applied and is within rate limits (at most 3 actions per tick, one high-impact action per AP per 120 s). A verified set is applied as a whole or not at all.

LLM roles. The LLM (a cloud model with an offline local fallback, temperature 0) has three roles and none can change the network. (a) It parses an operator's text into a `Policy` under a JSON schema (scope, objectives, hard constraints), with at most two repair attempts; a deterministic compiler turns the policy into bounded actions, which are verified like any other proposal. (b) A copilot answers questions by calling read-only tools and asking the twin. (c) A root-cause explainer turns an alert and a deterministic evidence summary into a structured diagnosis. Forecasting (Holt's linear smoothing [10] against a moving average) and anomaly detection (an Isolation Forest [11] on 30 s whole-network windows) support the heuristics and the explainer.

TABLE I
Compared controllers. V2 exists only to measure what the twin prevents.

V1 plain SDN	Controller's L2 learning only; stations re-associate by themselves.
V2 no twin	Heuristic proposals applied directly; the twin's verdict is computed and recorded but ignored.
V3 full	Proposals verified in the twin; high-impact ones wait for an operator; every change is watched and rolled back on regression.

VI. Baselines and Experimental Setup

Variants. The same testbed runs under three controllers (Table I). *Scenarios and seeds.* The three scenarios of Section III run with seeds 42, 43 and 44 under each variant: 27 runs of 600 s at one frozen code version. All data are *synthetic*, produced by the emulation; no real-network traces are used. The twin, forecaster and detector are calibrated on a separate no-control dataset of 12 runs (seeds 42–44 split by seed into training, validation and test).

Testbed and parameters. Mininet-WiFi with the Ryu controller (OpenFlow 1.3), 4 APs, 2 switches, 20 stations; per-AP downlink 4.6 Mbit/s. The loop runs every 5 s; the verifier tolerance is 5%, the executor's watch 30 s with a regression threshold of 10%. The campaign ran unattended: no operator approved high-impact actions. *Reproducibility.* The submission package contains the code, the recorded datasets, the seeds and the configuration; `make results` regenerates every table and figure of this section offline from the recorded runs, `make paper` rebuilds this paper's numbers from them, and `make campaign MODE=V1|V2|V3` re-runs one variant's nine runs on the testbed (about two hours each).

Metrics. For each run: time to recover (5) (a run that never recovers is counted at its end, so its mean is a lower bound); mean throughput and mean loss of the *affected flows* (those of the event AP's stations in the 30 s before the disruption, in the flash crowd the busiest AP's); p95 video latency of those flows; and Jain's index (4), averaged per 5 s over the stress

window. We report the mean over seeds with a 95% interval $\bar{x} \pm t_{0.975,n-1}\, s/\sqrt{n}$ ($t = 4.30$ for $n = 3$). H1 is called *met*, *not met* or *inconclusive*, the last where never-recovered runs could change the answer. An action is *harmful* if the watch measured a regression above 10% (rolled back, or the rollback failed).

VII. Results and Performance Analysis

A. Recovery and network KPIs (H1)

Table II and Fig. 2 give the results. In the *flash crowd*, plain SDN leaves the arriving students on ap1: the affected flows lose 43.9% of their packets and carry 0.20 Mbit/s. Both acting variants steer clients away, raising throughput to 0.52–0.53 Mbit/s and cutting loss to 9.2% (V2) and 8.5% (V3), with intervals far from V1's. Jain's index rises from 0.51 to 0.78 (V2) and 0.79 (V3), just under the 0.8 target. V2 and V3 are not distinguishable here: steering is applied without an operator in both. In the *AP failure*, the stations of ap2 re-associate by themselves, so V1 already works (4.8% loss) and acting adds little. The wide intervals and the p95 latency of V2 and V3 (348 and 672 ms against 18 ms) come from failed rollbacks: 3 of the 3 rollbacks that failed in the campaign were in AP-failure runs, where a steer issued shortly before the failure was judged by its 30 s watch against the failure itself, was rolled back onto the failed AP, and left a station without an AP for the rest of the run. In the *interference* scenario the fix is a high-impact channel change. V3's heuristics proposed it and the twin accepted it, but with no operator it was never applied; V3 could only steer clients, which still cut loss from 48.2% to 25.7%. V2 applied the change directly and recovered in every run (mean 146 s).

H1 is inconclusive in all three scenarios. V3 is faster than V1 by 8%, 17% and 0% (flash crowd, AP failure, interference; target 30%), but plain SDN never recovered in any of its nine runs (Fig. 3), so each V1 time is only a lower bound.

B. Safety and the twin's role (H2)

Table III gives what became of every action, from the executor ledgers of the passed runs. The twin rejected 3 actions that V2 applied anyway, 2 of which the watch then rolled back, and rejected 4 in V3, none of which reached the network: the twin does block actions that would have hurt, so **H2 holds**. But it blocked few. Most harmful actions were ones it had *accepted*: 20 in V2 and 18 in V3 were applied and then rolled back. The twin predicts the steady state after a change, not the transient in the first seconds, and only its throughput prediction was validated. The main protection was therefore the watch-and-rollback step, and it is not guaranteed: 3 of 117 applied actions could not be rolled back. In V3 every applied action had an accepted verdict and no high-impact change was applied without an operator (V2: 9 such changes), which is also why V3 could not repair the interference unattended; 8 accepted changes were still waiting for approval when their runs ended.

C. Twin accuracy, LLM and detection

Twin. On 586 held-out situations the twin's mean absolute percentage error for per-flow throughput is 4.5%: 4.0% in steady state, 3.3% after client steering and 8.1% after a channel change, while bulk TCP flows are weakest (12–21%). A prediction takes a median 0.61 ms. *LLM (H3).* On a 30-intent test set the cloud model translated 30 of 30 correctly (median 2.0 s) and the local fallback 21 of 30 (median 2.5 s), so **H3 holds**; every output passes the schema before the compiler sees it, and no invalid output reached the network. On live faults the copilot answered 5 of 5 diagnostic questions correctly, the explainer diagnosed 3 of 3 faults from the first alert, and all 6 of 6 LLM-proposed actions have their twin verdict in the audit log. *Detection.* The Isolation Forest reaches precision 0.950, recall 0.979 and F1 0.964 on the test split and flags the AP failure 5 s after it occurs. Holt's trend method does not beat the moving average (RMSE 0.129 against 0.117 at one minute, 0.256 against 0.212 at three), because load changes in steps; the loop therefore reacts to measured load rather than to the forecast.

VIII. Limitations

The network is emulated on one machine with a fixed propagation model, and the numbers describe this emulation, not a real campus. The twin is calibrated on this testbed and does not predict transients. With three seeds the intervals are wide, and the evaluation seeds reuse the seed numbers of the calibration dataset, though the runs are separate and the controller acts in them. The campaign was unattended, so V3 never applied a high-impact action; with an operator its behaviour would differ. Two fixes follow from the failures: a rollback should never target an AP that is down, and the watch should not judge an action while an outside event is unfolding.

IX. Conclusion

A digital twin used as a gate makes automated control of a Wi-Fi network safer, and an LLM behind that gate is a safe way to state goals in natural language: the model never acted on its own and no invalid output reached the network. Whether the loop also recovers the network faster depends on the fault and on having an operator for the changes that matter most; H1 could not be shown, and the evidence is mixed. We measured each failure instead of assuming it: the steers that hurt, the rollbacks that failed and the changes left waiting. Next steps are a rollback that avoids failed APs, a learned twin for bulk flows and transients, and an evaluation with an operator in the loop on real access points.

Acknowledgment

Parts of the code and of the text were drafted with the help of an AI coding assistant (Claude Code). The authors reviewed all content and take responsibility for it.

TABLE II

KPIs OF THE AFFECTED FLOWS (MEAN ± 95% INTERVAL OVER THREE SEEDS). ≥ MARKS A RECOVERY TIME THAT INCLUDES RUNS THAT NEVER RECOVERED (THEIR NUMBER IN BRACKETS), COUNTED AT THE RUN END.

Scenario	Variant	Recovery (s)	Thr. (Mbit/s)	p95 lat. (ms)	Loss (%)	Jain
Lecture flash crowd	V1 plain SDN	≥ 390 ± 0 (3)	0.20 ± 0.00	21 ± 1	43.9 ± 0.3	0.51 ± 0.00
	V2 no twin	≥ 374 ± 67 (2)	0.52 ± 0.03	24 ± 8	9.2 ± 4.9	0.78 ± 0.00
	V3 full	≥ 359 ± 101 (1)	0.53 ± 0.08	25 ± 9	8.5 ± 2.3	0.79 ± 0.04
AP failure	V1 plain SDN	≥ 360 ± 0 (3)	0.64 ± 0.09	18 ± 0	4.8 ± 1.0	0.96 ± 0.00
	V2 no twin	≥ 142 ± 470 (1)	0.84 ± 0.44	348 ± 1402	10.0 ± 32.5	0.91 ± 0.06
	V3 full	≥ 300 ± 258 (2)	0.58 ± 0.20	672 ± 1413	18.8 ± 31.4	0.89 ± 0.04
Co-channel interference	V1 plain SDN	≥ 420 ± 0 (3)	0.18 ± 0.00	11 ± 1	48.2 ± 0.4	0.93 ± 0.00
	V2 no twin	146 ± 445	0.60 ± 0.46	22 ± 7	9.1 ± 34.2	0.95 ± 0.05
	V3 full	≥ 420 ± 0 (3)	0.31 ± 0.03	11 ± 3	25.7 ± 7.5	0.94 ± 0.00

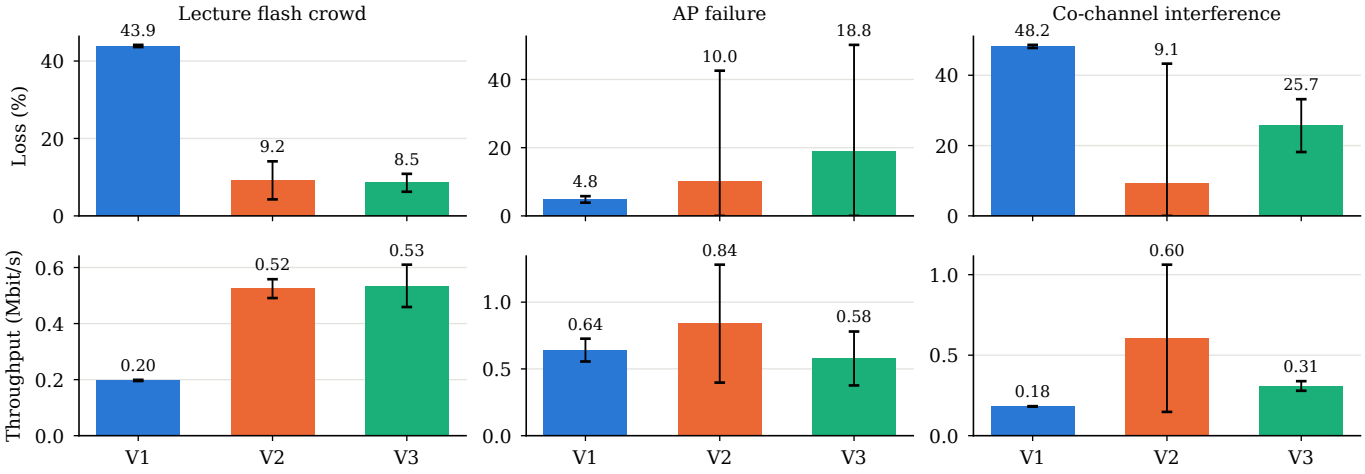


Fig. 2. Loss (top) and throughput (bottom) of the affected flows per scenario and variant; bars are means, whiskers the 95% interval over three seeds.

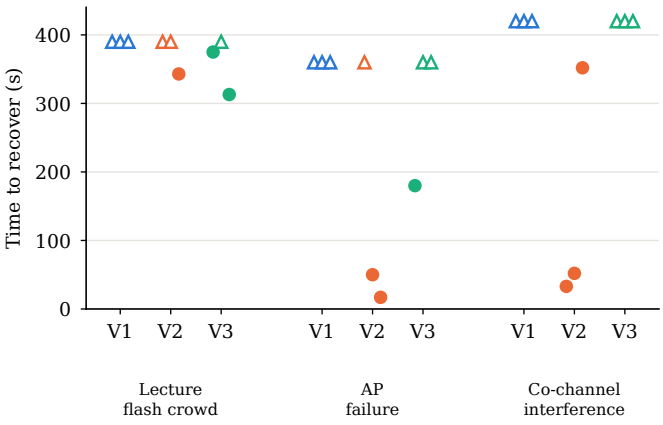


Fig. 3. Time to recover of every run. A hollow triangle is a run that never recovered, plotted at its run end.

TABLE III

FATE OF EVERY ACTION IN THE PASSED RUNS (NINE RUNS PER VARIANT), FROM THE EXECUTOR LEDGERS.

Actions	V1	V2	V3
Proposed	0	64	65
Kept	0	38	33
Rolled back	0	22	18
Rollback failed	0	1	2
Still watched	0	3	0
Rejected by the twin	0	0	4
Awaiting an operator	0	0	8

REFERENCES

[1] P. Almasan, M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, D. Perino, D. López, A. A. P. Perales, P. Harvey, L. Ciavaglia, L. Wong, V. Ram, S. Xiao, X. Shi, X. Cheng, A. Cabellos-Aparicio, and P. Barlet-Ros, "Network digital twin: Context, enabling technologies, and opportunities," *IEEE Commun. Mag.*, vol. 60, no. 11, pp. 22–27, Nov. 2022.

[2] K. Rusek, J. Suárez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 10, pp. 2260–2270, Oct. 2020.

[3] A. Mekrache, A. Ksentini, and C. Verikoukis, "Intent-based management of next-generation networks: an LLM-centric approach," *IEEE Netw.*, vol. 38, no. 5, pp. 29–36, Sep. 2024.

[4] L. Suresh, J. Schulz-Zander, R. Merz, A. Feldmann, and T. Vazão, "Towards programmable enterprise WLANs with Odin," in *Proc. 1st Workshop on Hot Topics in Software Defined Networks (HotSDN)*, Helsinki, Finland, 2012, pp. 115–120.

[5] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, Monterey,

CA, USA, Oct. 2010, pp. 19:1–19:6.
[6] R. R. Fontes, S. Afzal, S. H. B. Brito, M. A. S. Santos, and C. E. Rothenberg, "Mininet-WiFi: Emulating software-defined wireless networks," in *Proc. 11th Int. Conf. Netw. Service Manag. (CNSM)*, Barcelona, Spain, Nov. 2015, pp. 384–389.
[7] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Apr. 2008.
[8] R. Jain, D.-M. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Hudson, MA, USA, Tech. Rep. DEC-TR-301, Sep. 1984.
[9] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley, 1975.
[10] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *Int. J. Forecasting*, vol. 20, no. 1, pp. 5–10, 2004.
[11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.